

CS 103000

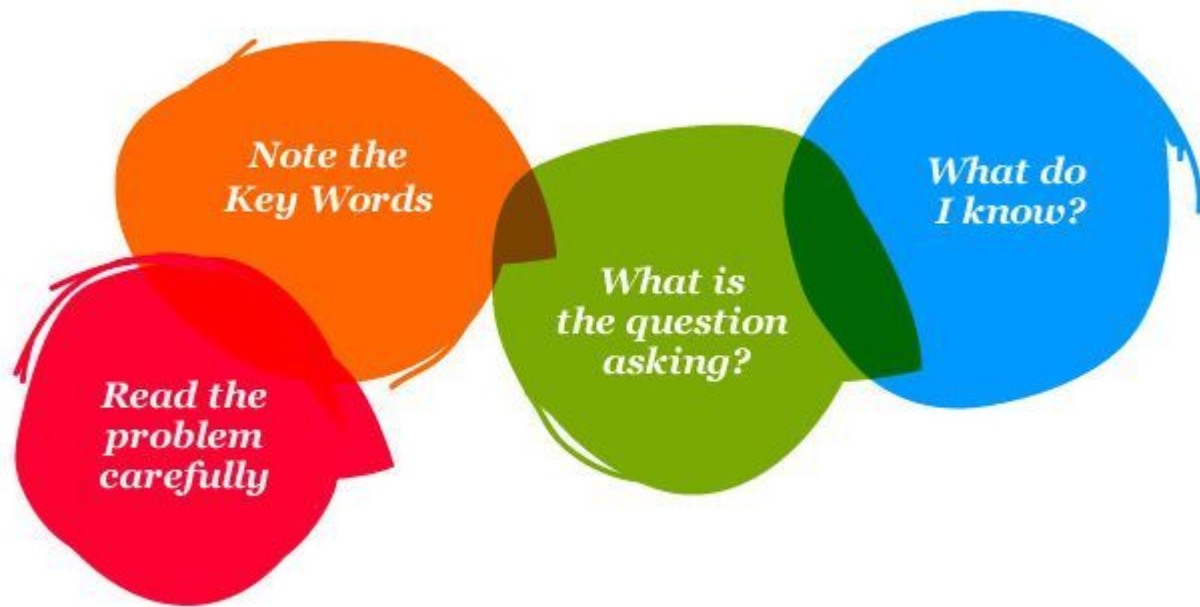
Prof. Madeline Blount

Week 9:

FUNCTIONS



Dall-E 2: cats learning C++ in the forest on '90's technology





Programming is like
writing a book...
Except when you miss
a single comma on
page 126
the whole thing
makes no sense.

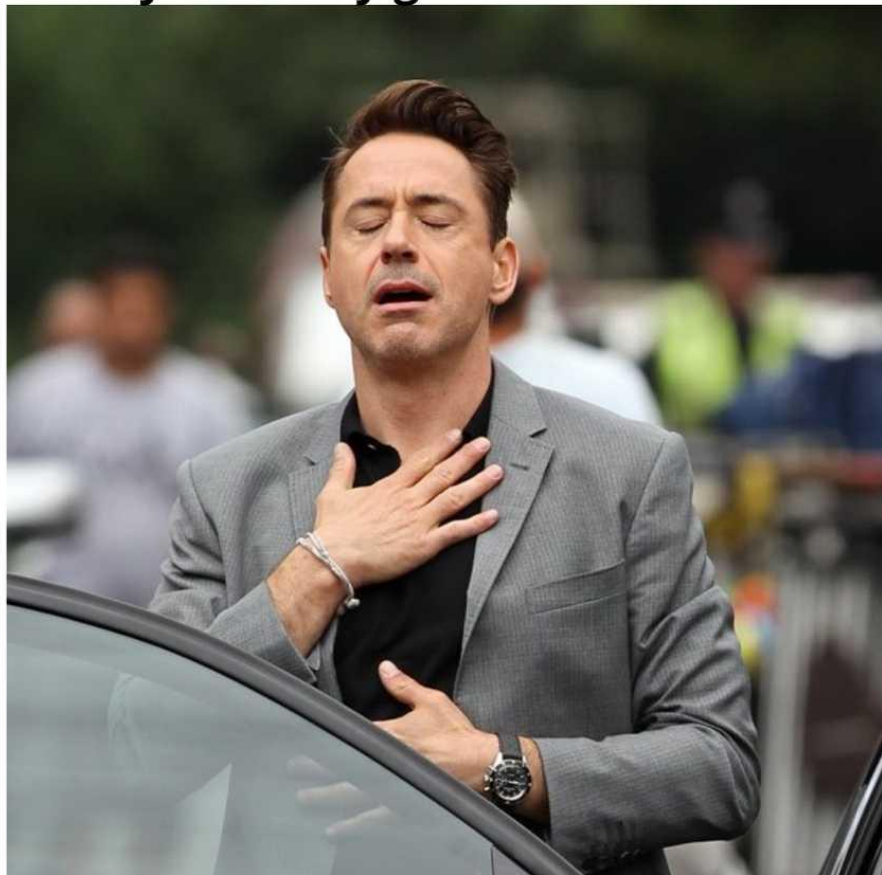
my program: *works perfectly*

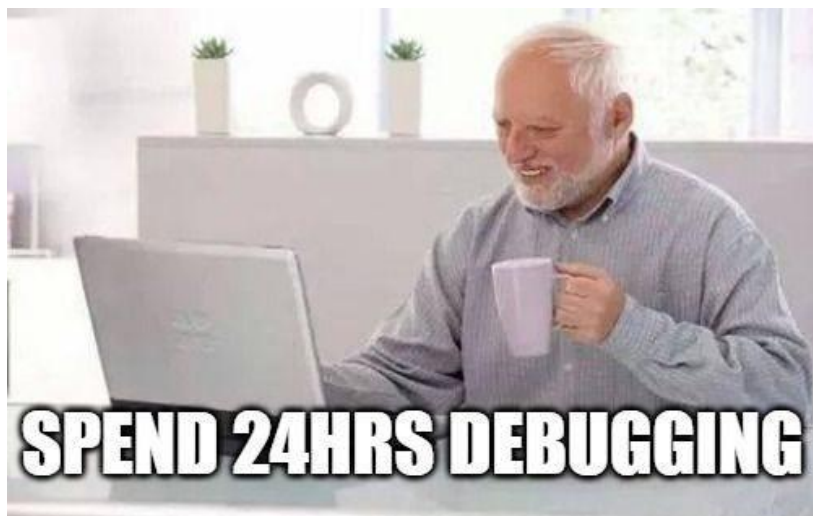
me: *cleans up the code*

also my program:

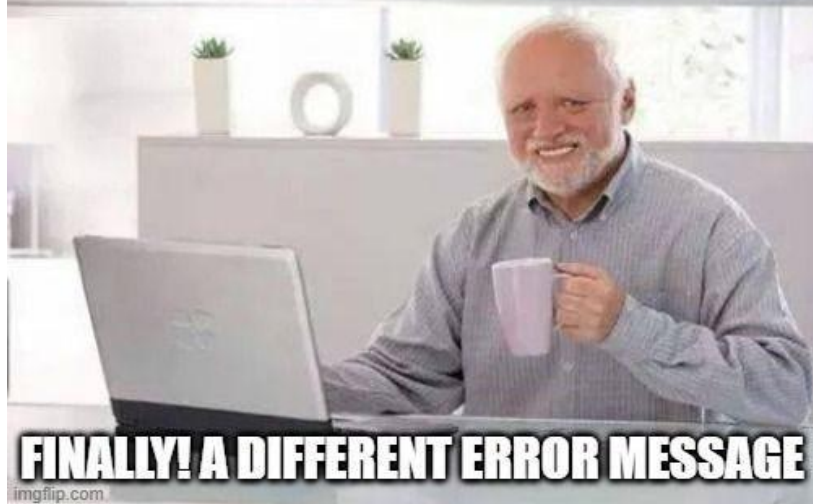


**When you stare at your code for 2 hours
and you finally get a different error**

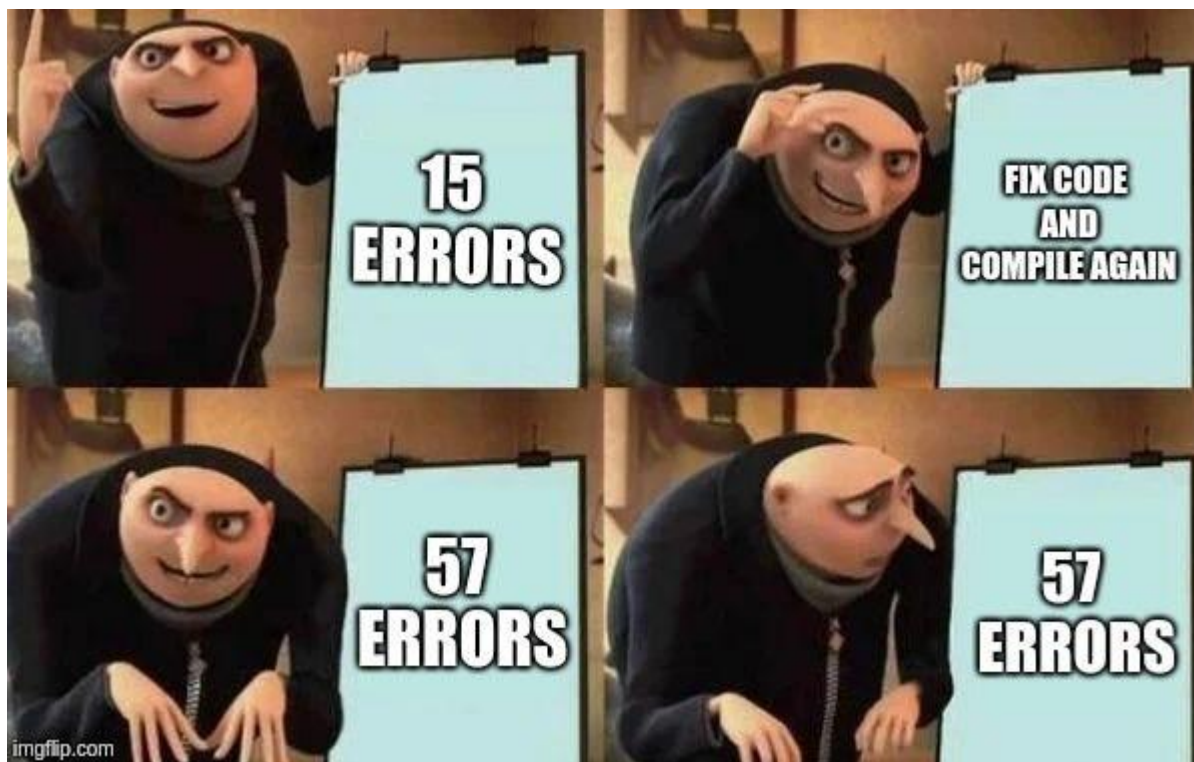




SPEND 24HRS DEBUGGING



FINALLY! A DIFFERENT ERROR MESSAGE

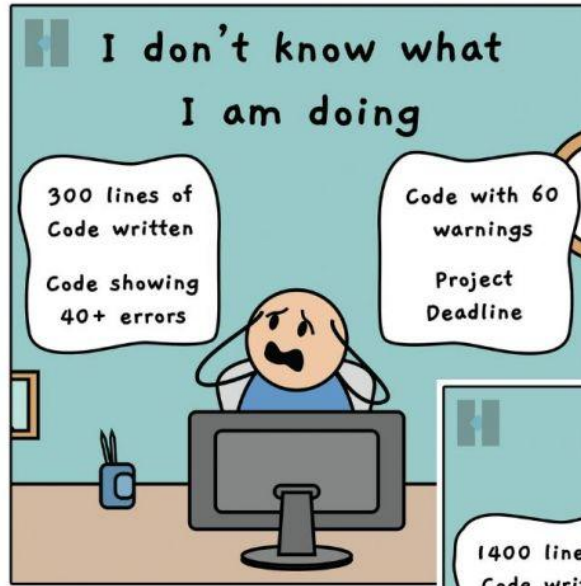


Error on line 42

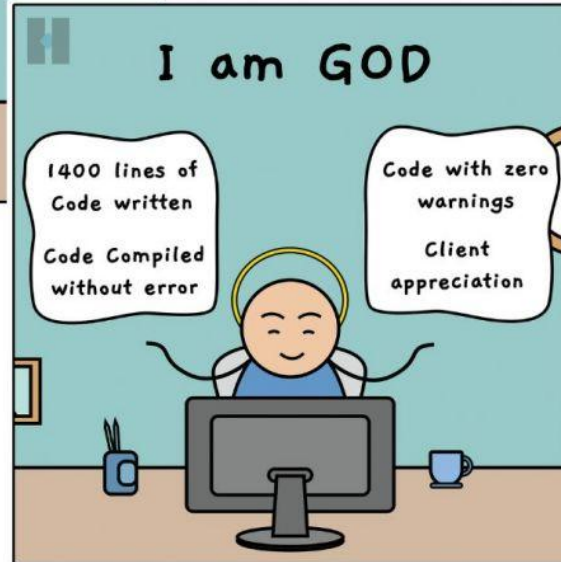
```
41      });  
42  
43      if (includ
```

imgflip.com

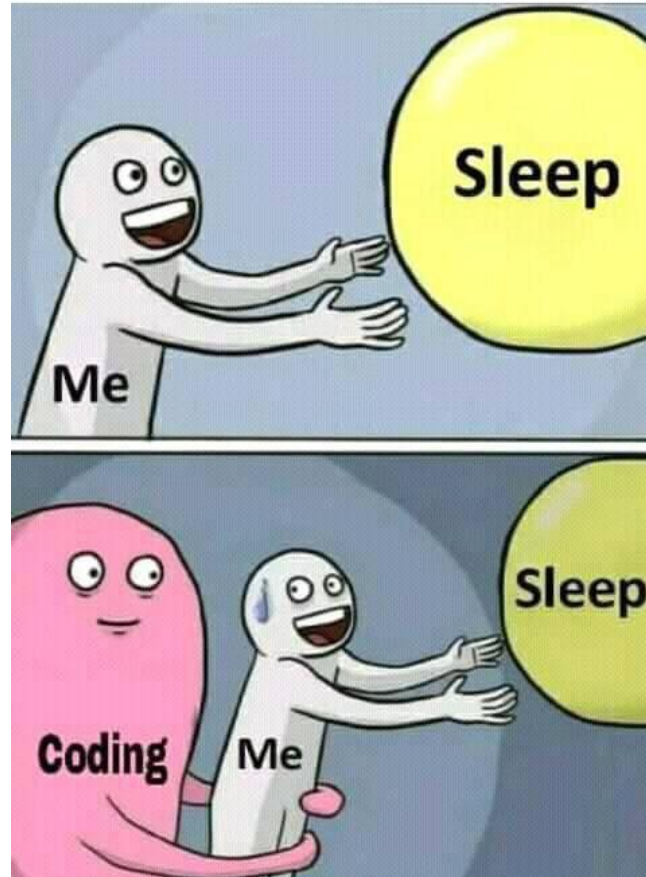




Two stages of Programmer



Me every night !!



AM I BROKEN ?

**I ACTUALLY ENJOY STAYING
UP ALL NIGHT LONG CODING**

I code better at night

11:00 PM



11:05 PM



AM I TESTING THIS PROGRAM



OR IS IT TESTING ME?

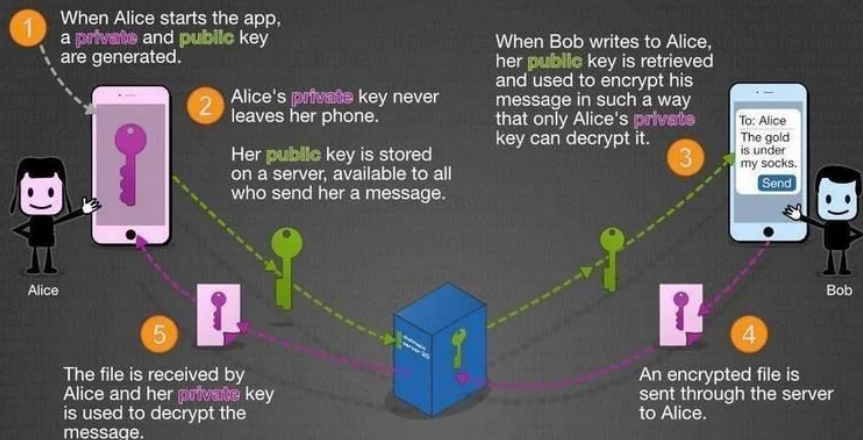
STAND BACK

WE'RE TRYING THIS IN PRODUCTION

ONE DOES NOT SIMPLY

ROLL THEIR OWN CRYPTO

End-to-End Encryption Explained



Prime Numbers & Encryption

$$11 \times 17 = 187$$

The product of 2 large random prime numbers is the backbone of encryption.



Cracking the encryption means figuring out the 2 factors. Using brute-force, it takes decades with today's computers. If the 2 numbers are known (a **private** key), a split second is all it takes.



$$17,425,170$$

The number of *digits* in the largest known prime number.



The **public** key is made up in part by calculating the number of integers that share no common factors, that are less than the product of the 2 prime numbers (encryption is supposed to be confusing).







USER-DEFINED FUNCTIONS!

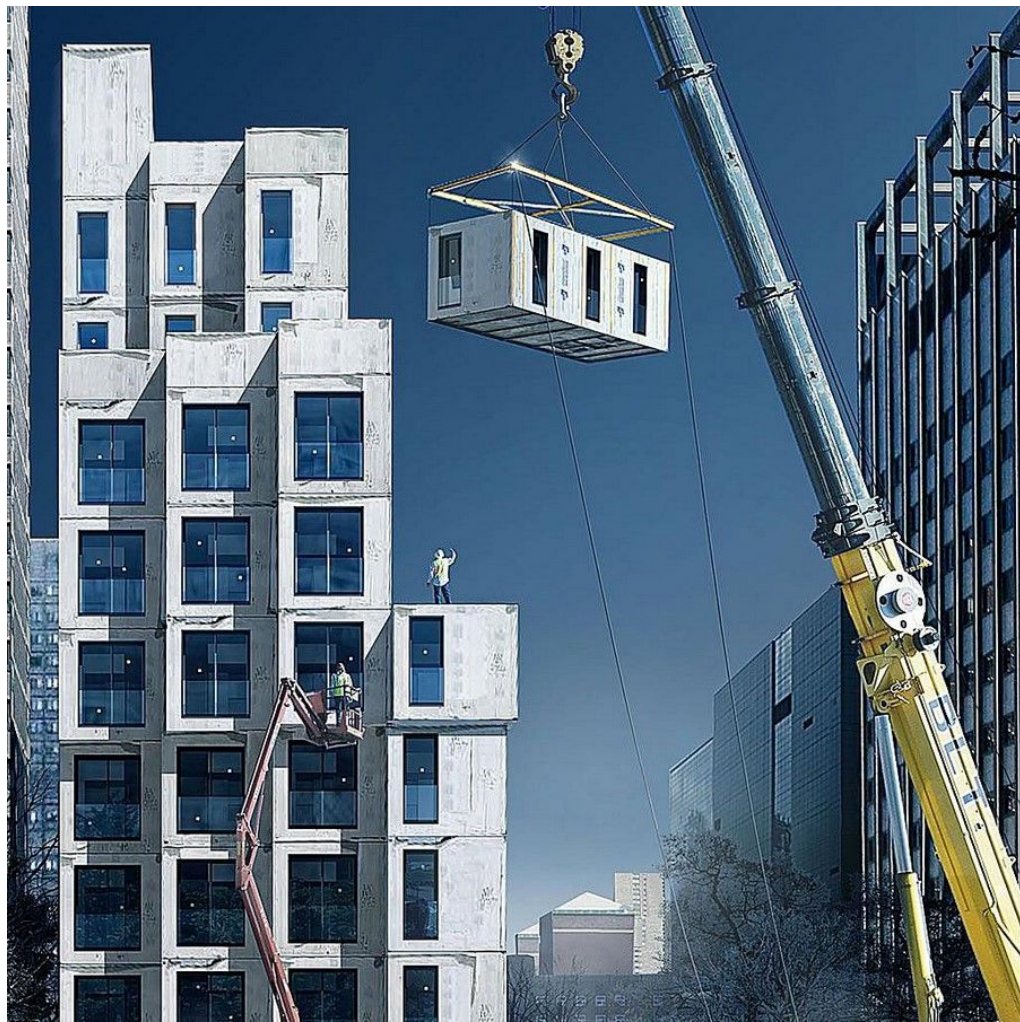
- Building blocks of code
- Set of executable statements (runs when called)

WHY?

- Easier to read (style)
- Easier to debug
- Easier to reuse, not repeat code
- Modular







Function structure:

- **1. Declare**
(above main)
- **2. Define**
(below main)
- **3. Call**
(inside main)

```
#include <iostream>

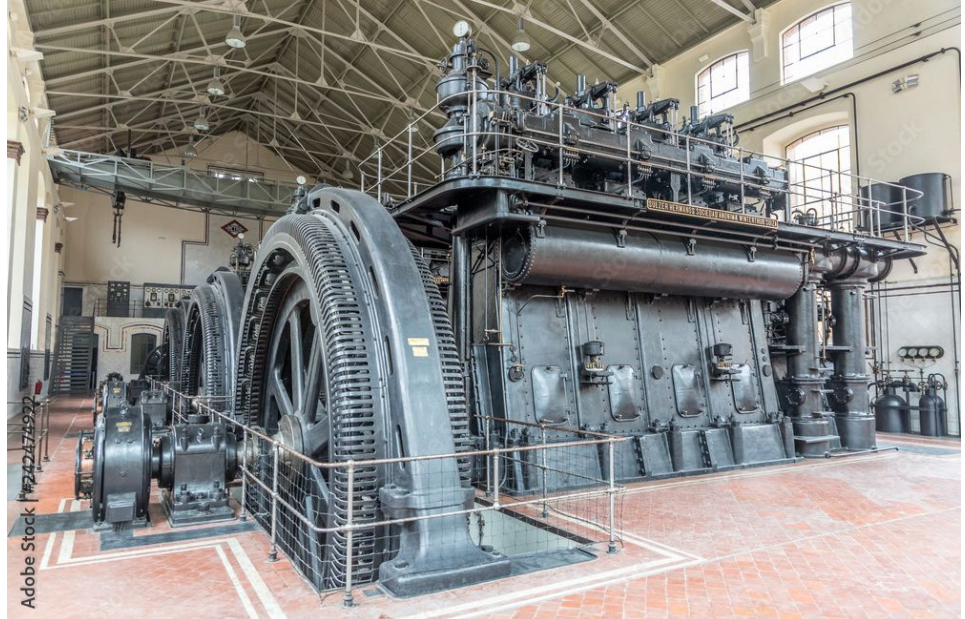
int sum(int a, int b);

int main() {
    int r = sum(10, 20);
    std::cout << r;
}

int sum(int a, int b) {
    return(a + b);
}
```

FUNCTIONS: abstraction

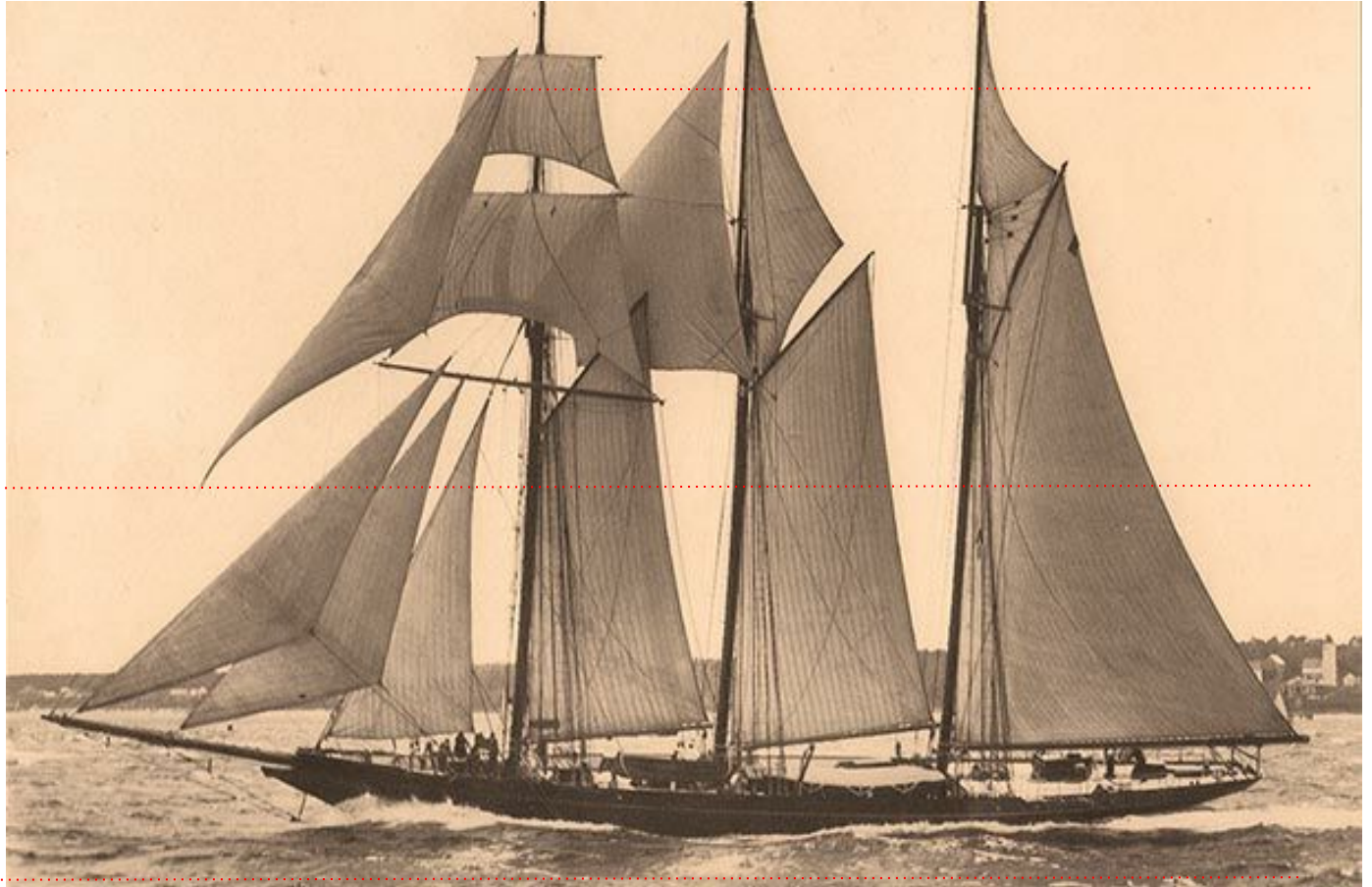
farther **away** from the guts, computation, data, hardware, etc.



high-level

mid-level

low-level





Function declaration anatomy:

return type name(parameter list);

```
int myFunction(int var1, int var2);
```




Function definition anatomy:

return type name(parameter list)

```
int myFunction(int var1, int var2)
{
    // my code!
    return something
}
```



Function call anatomy:

name(arguments);

```
myFunction(num1, num2);
```



VOID function anatomy:

return type name(parameter list)

```
void sayHello(string name) {  
    // my code!  
    cout << "hello, " << name;  
}
```

```
#include <iostream>

// Declaring a function
void print();

int main() {
    print();
}

// Defining a function
void print() {
    std::cout << "Hello World!";
}
```




MAIN function anatomy:

return type name(parameter list)

Return type = int!

0 = exit code, “success”

```
int main() {  
    // my code!  
    return 0  
}
```



SCOPE

GLOBAL
everywhere!

local
only accessible in the function where they were created



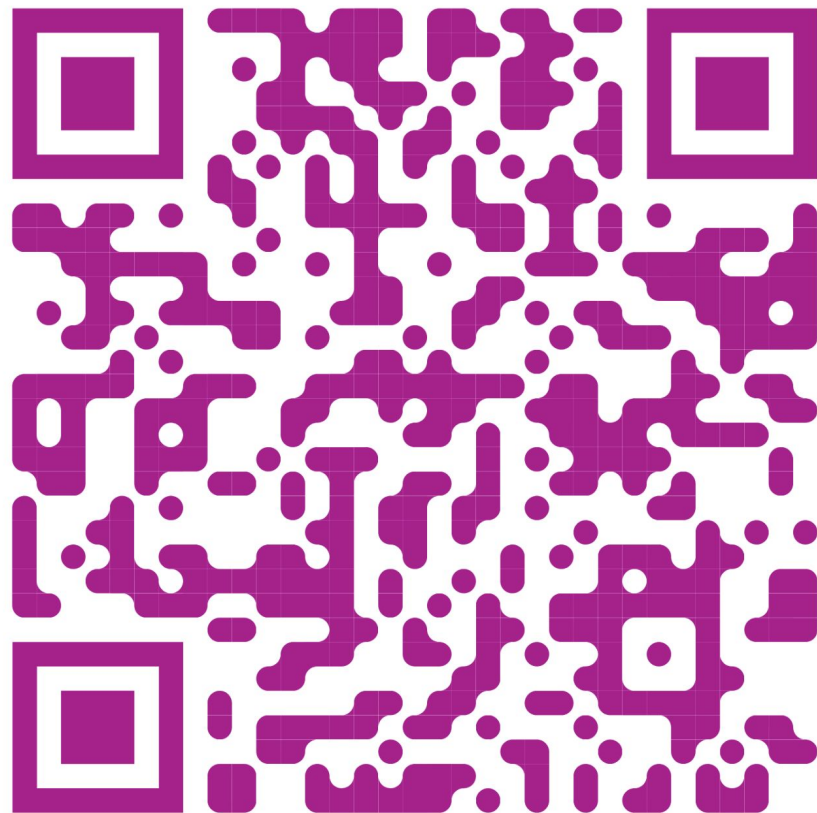
```
#include <iostream>
```

```
void print();
```

```
int i = 10;      // global variable
```

```
int main() {  
    std::cout << i << "\n";  
}
```

```
void print() {  
    int j = 0;    // local variable  
    i = 20;  
    std::cout << i << "\n";  
    std::cout << j << "\n";  
}
```



<https://shorturl.at/koPfL>

"The Analytical Engine has no pretensions whatever to originate anything. It can do whatever we know how to order it to perform. It can follow analysis but it has no power of anticipating any analytical relations or truths Its province is to assist us in making available what we are already acquainted with ... The Analytical Engine weaves algebraic patterns just as the Jacquard loom weaves flowers and leaves."

**Humans saw this
and decided to invent C++**





FILE STRUCTURE



main.cpp

#include



Function
declarations

myheader.h



Function
definitions

myfunctions.cpp

compile: `g++ main.cpp myfunctions.cpp`

"Function overload":

Functions can have the same name but handle different data types

```
int cubeNumber(int x);  
double cubeNumber(double x);
```

"PASS BY VALUE":

everything we have done so far, local variables stay local, because every parameter is a "copy"

"PASS BY REFERENCE":

can treat parameter like a global variable, changes outside of scope

pass by reference



fillCup()

pass by value



fillCup()